

Project Report

Library Management System

Course: Software Construction & Design

Submission Date: May 2026

Status: Completed MVP

Contents

Introduction	2
User Interface & Layout Design	2
Absolute Layout Paradigm	2
Alignment and Grid Spacing	2
Architectural Validation Engine	2
Lenient Temporal Parsing	2
Exception Handling Infrastructure	3
Conclusion	3

Introduction

This document outlines the design, layout philosophy, and architectural implementation of a lightweight desktop-based **Library Management System**. Built using Java's foundational UI toolkit, `javax.swing`, the application streamlines the administrative workflow of documenting book issuances to university students.

The primary scope of this construction project was to transition from manual entry checking toward a programmatic validation engine that respects chronological boundaries while preserving structural integrity.

User Interface & Layout Design

Absolute Layout Paradigm

To ensure fine-grained control over UI element placements and maintain absolute pixel fidelity, the interface rejects layout managers in favor of an explicit absolute coordinate space (`ContentPane.setLayout(null)`). This guarantees that the visual rendering remains perfectly identical across runtime host environments and retains structured compatibility with design canvas tooling like **Eclipse's WindowBuilder**.

Alignment and Grid Spacing

The visual elements are strictly anchored onto a standardized horizontal grid alignment:

- **Labels:** Left-aligned at coordinate $X = 45$ with a unified width of 140 pixels.
- **Input Fields:** Left-aligned at coordinate $X = 200$ stretching to a maximum bounds width of 245 pixels.
- **Vertical Padding:** Each sequential entry row maintains a uniform vertical bounding displacement delta of 50 pixels, creating an aesthetically predictable scannable flow.

Architectural Validation Engine

Lenient Temporal Parsing

A major vulnerability in standard date parsers is the overhead of parsing real-world calendar constraints concurrently with basic syntactic matching. This application

addresses this by leveraging Java's built-in `DateTimeFormatter` paired explicitly with `ResolverStyle.LENIENT`.

This structural parsing design isolates syntax verification from calendar validation:

1. It validates that the string structure strictly complies with the desired `dd/MM/yyyy` footprint.
2. It prevents abrupt application termination or crash overflows during parsing operations.
3. Chronological safety constraints are executed smoothly using the built-in native `.isBefore()` method to assert that a target return date does not precede the initial issuance timestamp.

Exception Handling Infrastructure

The application implements a localized custom exception hierarchical chain to isolate input anomalies. Errors do not quietly fail; instead, they explicitly map back to distinct modal warnings:

Custom Exception Class	Validation Trigger Condition
<code>EmptyFieldException</code>	Fires if text blocks are blank or retain default prompt helpers.
<code>InvalidRollNumberException</code>	Enforces numeric-only constraints on student identifiers.
<code>NullSelectionException</code>	Validates drop-down combo-boxes and radio selections.
<code>InvalidDateException</code>	Fires when chronological order sequences are inverted.

Conclusion

The resulting artifact successfully demonstrates a clean, robust, simple form-submission architecture. By decoupling structural layout dependencies and isolating parsing logic using built-in native Java capabilities, the application satisfies the requirements of a stable desktop interface.